React Component Lifecycle

In React, a component has 3 main life stages:

```
1. Birth  → Mounting
2. Life   → Updating
3. Death  → Unmounting
```

Think of a component like a human.

---

1. Mounting (Birth)

Component is created and shown on screen for the FIRST time.

Example:

```
function App() {
  return <h1>Hello</h1>;
}
```

When React shows this on screen:

```
Component is mounted
```

Real Life Example

Imagine opening Instagram.

When homepage appears:

- components are created
- buttons appear
- posts appear
- navbar appears

This is:

```
Mounting
```

During Mounting React Does:

```
1. Calls component function
2. Creates HTML
3. Adds it to DOM
4. Shows on screen
```

Visual Flow

```
Component Created
       ↓
```

```
Function Runs
      ↓
HTML Generated
      ↓
Added to Screen
```

Example with useEffect

```
useEffect(() => {
  console.log("Component mounted");
}, []);
```

Runs only once after mounting.

---

2. Updating (Life)

Whenever:

- state changes
- props change

React updates the component.

Example

```
const [count, setCount] = useState(0);
```

Click button:

```
setCount(count + 1);
```

React updates UI.

What Happens Internally?

```
State changes
      ↓
Component function runs again
      ↓
New HTML created
      ↓
React compares old vs new
      ↓
Only changed parts update
```

Example

Before:

```
<h1>0</h1>
```

After clicking:

```
<h1>1</h1>
```

React updates ONLY text.

Not full page reload.

Real Life Example

Instagram likes increase.

Only:

```
Like count updates
```

Entire page does NOT reload.

This is React updating.

---

useEffect During Updating

```
useEffect(() => {
  console.log("Count changed");
}, [count]);
```

Runs whenever count changes.

---

3. Unmounting (Death)

When component disappears from screen.

Example:

```
{show && <Sidebar />}
```

If:

```
setShow(false)
```

Sidebar removed.

This is:

```
Unmounting
```

Real Life Example

Closing:

- modal
- popup
- sidebar

- page

Components are destroyed.

During Unmounting React Does:

```
1. Removes component from DOM
2. Clears memory
3. Runs cleanup functions
```

Cleanup Example

```
useEffect(() => {
  const id = setInterval(() => {
    console.log("Running");
  }, 1000);

  return () => {
    clearInterval(id);
  };
}, []);
```

When component removed:

```
cleanup runs
```

Interval stops.

Complete Lifecycle Simple Diagram

```
        MOUNTING
   (Component appears)
            ↓
        UPDATING
 (State/Props change)
            ↓
      UNMOUNTING
(Component disappears)
```

Very Simple Real-Life Analogy

Mobile App Screen

Mounting

You open WhatsApp chat screen.

Screen appears.

Updating

New messages arrive.

Chat updates.

Unmounting

You close chat screen.

Screen disappears.

Super Important Understanding

React component is basically:

```
A function that React runs again and again
```

Example

```
function App() {
  return <h1>Hello</h1>;
}
```

React keeps running this function whenever needed.

Simple Internal Flow

```
State changes
      ↓
React reruns component
      ↓
Creates new UI
      ↓
Updates screen
```

Easiest Way to Remember

Stage	Meaning
Mounting	Component created
Updating	Component changed
Unmounting	Component removed

Tiny Example Covering All Lifecycle

```
function App() {
  const [count, setCount] = useState(0);
```

```
  useEffect(() => {
    console.log("Mounted");

    return () => {
      console.log("Unmounted");
    };
  }, []);

  useEffect(() => {
    console.log("Updated");
  }, [count]);

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

Output Flow

First render:

```
Mounted
Updated
```

Click button:

```
Updated
```

Component removed:

```
Unmounted
```

Final Simple Definition

React component lifecycle means:

```
How a component is created,
updated,
and removed from the screen.
```